

OCS2 Legged Robot MPC: Mathematical Problem Formulation

Note based on analysis of `ocs2_legged_robot` codebase

July 20, 2026

Contents

1	Overview	3
2	State and Input Variables	3
2.1	State Vector	3
2.2	Input Vector	3
3	Dynamics: Single Rigid Body Model (SRBD)	4
3.1	Centroidal Dynamics Equations	4
3.1.1	Normalized Linear Momentum Rate	4
3.1.2	Normalized Angular Momentum Rate	4
3.1.3	Base Pose Rate	4
3.1.4	Joint Rate	4
3.2	SRBD Approximation	4
3.3	Contact Point Kinematics	5
4	The Optimal Control Problem	5
4.1	Problem Statement	5
4.2	Cost Function	5
4.2.1	Running Cost	5
4.2.2	Terminal Cost	6
5	Constraints	6
5.1	Friction Cone Constraint	6
5.1.1	Soft Constraint Formulation (Default)	7
5.2	Zero Force Constraint (Swing Phase)	7
5.3	Zero Velocity Constraint (Stance Phase)	7
5.4	Normal Velocity Constraint (Swing Phase)	8
6	Constraint Activation Summary	8
7	Frontend: Gait Scheduling	8
7.1	Contact Mode Encoding	8
7.2	Gait Definition	8
7.3	Mode Schedule Generation	9
7.4	Reference Manager	9
8	Frontend: Swing Trajectory Planning	9
8.1	Swing Phase Identification	10
8.2	Vertical Trajectory Construction	10

9 Frontend: Pre-Computation and Dynamic Constraint Updates	10
10 Solver Configuration	11
10.1 Multiple Shooting Transcription	11
10.2 SQP Solver (Default)	11
10.3 Alternative Solvers	11
11 Complete Discretized OCP	11
12 Summary of the Full Pipeline	12
13 Key Design Decisions and Notes	12

1 Overview

OCS2 legged robot MPC solves a **switched-system nonlinear optimal control problem (OCP)** in receding horizon fashion. The system is a quadruped robot with 4 legs, each having 3 actuated joints (12 joints total).

The key architectural choice is:

- **Dynamics model:** Centroidal dynamics (specifically, Single Rigid Body Dynamics — SRBD).
- **Contact handling:** *Not* contact-implicit. Instead, contact mode sequences (which feet are in stance/swing) are *predetermined* by a gait scheduler (frontend). The MPC solves the OCP *conditioned on* this given mode schedule.
- **Solver:** Multiple-shooting SQP (or DDP/IPM as alternatives), running at 50 Hz with a 1-second horizon.

This document describes (1) the MPC problem formulation in detail, and (2) the frontend components that feed into the MPC.

2 State and Input Variables

2.1 State Vector

The state $\mathbf{x} \in \mathbb{R}^{6+6+n_j}$ is:

$$\mathbf{x} = \begin{bmatrix} \mathbf{h} \\ \mathbf{q}_b \\ \mathbf{q}_j \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{p}}_{\text{com}} \\ \boldsymbol{\omega}_{\text{com}} \\ \mathbf{r}_b \\ \boldsymbol{\theta}_{zyx} \\ \mathbf{q}_j \end{bmatrix} \in \mathbb{R}^{24} \quad (1)$$

where:

- $\mathbf{h} \in \mathbb{R}^6$: **Normalized centroidal momentum**, consisting of:
 - $\dot{\mathbf{p}}_{\text{com}} = \frac{1}{m}\mathbf{l} \in \mathbb{R}^3$: Normalized linear momentum (= CoM velocity)
 - $\boldsymbol{\omega}_{\text{com}} = \frac{1}{m}\mathbf{k} \in \mathbb{R}^3$: Normalized angular momentum
- $\mathbf{r}_b \in \mathbb{R}^3$: Base position in world frame $[x, y, z]^\top$
- $\boldsymbol{\theta}_{zyx} \in \mathbb{R}^3$: Base orientation as ZYX Euler angles $[\theta_z, \theta_y, \theta_x]^\top$
- $\mathbf{q}_j \in \mathbb{R}^{12}$: Joint angles for 4 legs $\times$ 3 joints each

The joint ordering is:

$$\mathbf{q}_j = [q_{\text{LF_HAA}}, q_{\text{LF_HFE}}, q_{\text{LF_KFE}}, q_{\text{LH_HAA}}, \dots, q_{\text{RF_HAA}}, \dots, q_{\text{RH_HAA}}, \dots, q_{\text{RH_KFE}}]^\top$$

2.2 Input Vector

The input $\mathbf{u} \in \mathbb{R}^{12+12}$ is:

$$\mathbf{u} = \begin{bmatrix} \mathbf{f}_c \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{24} \quad (2)$$

where:

- $\mathbf{f}_c \in \mathbb{R}^{12}$: Contact forces at 4 feet, each $\mathbf{f}_i \in \mathbb{R}^3$ in the world frame:

$$\mathbf{f}_c = [\mathbf{f}_{\text{LF}}^\top, \mathbf{f}_{\text{RF}}^\top, \mathbf{f}_{\text{LH}}^\top, \mathbf{f}_{\text{RH}}^\top]^\top$$

- $\dot{\mathbf{q}}_j \in \mathbb{R}^{12}$: Joint velocities (same ordering as $\mathbf{q}_j$)

Note that the input contains *contact forces* (not joint torques). The mapping from contact forces and joint velocities to actual joint torques is handled downstream by a whole-body controller (not part of this MPC).

3 Dynamics: Single Rigid Body Model (SRBD)

3.1 Centroidal Dynamics Equations

The dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ are decomposed into four blocks:

3.1.1 Normalized Linear Momentum Rate

$$\ddot{\mathbf{p}}_{\text{com}} = \mathbf{g} + \frac{1}{m} \sum_{i \in \mathcal{C}(t)} \mathbf{f}_i \quad (3)$$

where $\mathbf{g} = [0, 0, -9.81]^\top$ is gravity and $\mathcal{C}(t)$ is the set of active contact feet at time t .

3.1.2 Normalized Angular Momentum Rate

$$\dot{\boldsymbol{\omega}}_{\text{com}} = \frac{1}{m} \sum_{i \in \mathcal{C}(t)} \mathbf{r}_i \times \mathbf{f}_i \quad (4)$$

where $\mathbf{r}_i = \mathbf{p}_{\text{foot},i} - \mathbf{p}_{\text{com}}$ is the position vector from the center of mass to the i -th contact point, computed via forward kinematics.

3.1.3 Base Pose Rate

$$\dot{\mathbf{r}}_b = \dot{\mathbf{p}}_{\text{com}} \quad (5)$$

$$\dot{\boldsymbol{\theta}}_{zyx} = \mathbf{W}_{zyx}^{-1}(\boldsymbol{\theta}_{zyx}) \boldsymbol{\omega}_{\text{com}} \quad (6)$$

where $\mathbf{W}_{zyx}$ is the mapping from ZYX Euler angle rates to angular velocity:

$$\boldsymbol{\omega} = \mathbf{W}_{zyx}(\boldsymbol{\theta}_{zyx}) \dot{\boldsymbol{\theta}}_{zyx} \quad (7)$$

Note: Eq. (5) is an approximation — strictly, $\dot{\mathbf{r}}_b \neq \dot{\mathbf{p}}_{\text{com}}$ because the base frame origin and CoM generally differ. The SRBD model assumes the offset $\mathbf{p}_{\text{com}} - \mathbf{r}_b$ is constant at the nominal configuration.

3.1.4 Joint Rate

$$\dot{\mathbf{q}}_j = \dot{\mathbf{q}}_j \quad (\text{input directly}) \quad (8)$$

3.2 SRBD Approximation

The “Single Rigid Body Dynamics” model (`centroidalModelType = 1`) makes a key simplification: the centroidal momentum matrix $\mathbf{A}_g$ relating generalized velocities to centroidal momentum is computed using the *nominal* (default) joint configuration rather than the current configuration:

$$\mathbf{A}_g^{\text{SRBD}} = \begin{bmatrix} m\mathbf{I}_3 & m [\mathbf{p}_{\text{com}}^0]_{\times} \mathbf{W}_{zyx} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{\text{com}}^0 \mathbf{W}_{zyx} & \mathbf{0} \end{bmatrix} \quad (9)$$

where $\mathbf{p}_{\text{com}}^0$ is the nominal CoM-to-base offset and $\mathbf{I}_{\text{com}}^0$ is the nominal centroidal inertia. This avoids recomputing the full centroidal inertia at every time step (as required by the Full Centroidal Dynamics variant, `centroidalModelType = 0`).

3.3 Contact Point Kinematics

The position of the i -th foot in the world frame is computed via Pinocchio forward kinematics:

$$\mathbf{p}_{\text{foot},i} = \text{FK}_i(\mathbf{r}_b, \boldsymbol{\theta}_{zyx}, \mathbf{q}_j) \quad (10)$$

The Jacobian $\mathbf{J}_i = \frac{\partial \mathbf{p}_{\text{foot},i}}{\partial \mathbf{q}_{\text{gen}}}$ maps generalized velocities to foot velocities:

$$\dot{\mathbf{p}}_{\text{foot},i} = \mathbf{J}_i(\mathbf{q}_{\text{gen}}) \dot{\mathbf{q}}_{\text{gen}} \quad (11)$$

where $\mathbf{q}_{\text{gen}} = [\mathbf{r}_b^\top, \boldsymbol{\theta}_{zyx}^\top, \mathbf{q}_j^\top]^\top \in \mathbb{R}^{18}$ is the Pinocchio generalized coordinate vector.

4 The Optimal Control Problem

4.1 Problem Statement

The MPC solves the following OCP at every control cycle:

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \quad & \int_{t_0}^{t_0+T} \ell(\mathbf{x}(t), \mathbf{u}(t), t) dt + \Phi(\mathbf{x}(t_0 + T)) \\ \text{s.t.} \quad & \dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \\ & \mathbf{g}_{\text{eq}}^{(\sigma(t))}(\mathbf{x}(t), \mathbf{u}(t)) = \mathbf{0}, \\ & \mathbf{g}_{\text{ineq}}^{(\sigma(t))}(\mathbf{x}(t), \mathbf{u}(t)) \geq \mathbf{0}, \\ & \mathbf{x}(t_0) = \hat{\mathbf{x}}(t_0), \end{aligned} \quad (12)$$

where:

- $T = 1.0$ s is the prediction horizon
- $\sigma(t) \in \{0, 1, \dots, 15\}$ is the **contact mode** at time t , given by the gait scheduler (Section 7)
- Equality and inequality constraints are **mode-dependent**: different constraints are active depending on which feet are in stance vs. swing
- The dynamics f are mode-independent (Section 3)

4.2 Cost Function

4.2.1 Running Cost

$$\ell(\mathbf{x}, \mathbf{u}, t) = \|\mathbf{x} - \mathbf{x}^{\text{ref}}(t)\|_{\mathbf{Q}}^2 + \|\mathbf{u} - \mathbf{u}^{\text{ref}}(t)\|_{\mathbf{R}}^2 + \sum_{i \in \mathcal{C}(t)} p_{\text{friction},i}(\mathbf{u}) \quad (13)$$

where $\|\mathbf{v}\|_{\mathbf{M}}^2 = \mathbf{v}^\top \mathbf{M} \mathbf{v}$.

State Weight Matrix $\mathbf{Q}$. $\mathbf{Q} = \text{diag}(\mathbf{q})$ with:

$$\mathbf{q} = [\underbrace{15, 15, 30}_{\mathbf{p}_{\text{com}}}, \underbrace{5, 10, 10}_{\boldsymbol{\omega}_{\text{com}}}, \underbrace{500, 500, 500}_{\mathbf{r}_b}, \underbrace{100, 200, 200}_{\boldsymbol{\theta}_{zyx}}, \underbrace{20, \dots, 20}_{12 \text{ joints}}] \quad (14)$$

Input Weight Matrix $\mathbf{R}$. The input cost is defined in a task-space representation and then transformed to joint space. Define the task-space weight:

$$\mathbf{R}_{\text{task}} = 10^{-3} \cdot \text{diag}(\underbrace{[1, \dots, 1]}_{12 \text{ forces}}, \underbrace{[5000, \dots, 5000]}_{12 \text{ foot velocities}}) \quad (15)$$

The actual input weight $\mathbf{R}$ is:

$$\mathbf{R} = \begin{bmatrix} \mathbf{R}_{\text{task}}^{ff} & \mathbf{0} \\ \mathbf{0} & \mathbf{J}_{\text{feet}}^{0\top} \mathbf{R}_{\text{task}}^{vv} \mathbf{J}_{\text{feet}}^0 \end{bmatrix} \quad (16)$$

where $\mathbf{J}_{\text{feet}}^0 \in \mathbb{R}^{12 \times 12}$ is the stacked foot-to-joint Jacobian evaluated at the nominal configuration. This transforms foot-velocity penalties into joint-velocity penalties.

State Reference $\mathbf{x}^{\text{ref}}(t)$. Generated by the target trajectory module from user commands (desired velocity, heading, CoM height):

$$\mathbf{x}^{\text{ref}}(t) = [\dot{\mathbf{p}}_{\text{com}}^{\text{ref}\top}, \boldsymbol{\omega}^{\text{ref}\top}, \mathbf{r}_b^{\text{ref}\top}(t), \boldsymbol{\theta}^{\text{ref}\top}(t), \mathbf{q}_j^{\text{ref}\top}]^\top \quad (17)$$

Input Reference $\mathbf{u}^{\text{ref}}(t)$ — Weight Compensation. The nominal input distributes the robot's weight equally among stance feet:

$$\mathbf{u}^{\text{ref}}(t) = \begin{bmatrix} \mathbf{f}_c^{\text{ref}} \\ \mathbf{0}_{12} \end{bmatrix}, \quad \mathbf{f}_i^{\text{ref}} = \begin{cases} \begin{bmatrix} 0 \\ 0 \\ \frac{mg}{|\mathcal{C}(t)|} \end{bmatrix} & \text{if } i \in \mathcal{C}(t) \\ \mathbf{0}_3 & \text{if } i \notin \mathcal{C}(t) \end{cases} \quad (18)$$

where $|\mathcal{C}(t)|$ is the number of stance legs at time t .

Friction Cone Penalty $p_{\text{friction},i}(\mathbf{u})$. When using soft constraints (default), the friction cone is enforced via a relaxed log-barrier penalty (Section 5.1.1).

4.2.2 Terminal Cost

$$\Phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}^{\text{ref}}(t_0 + T)\|_{\mathbf{Q}}^2 \quad (19)$$

5 Constraints

All constraints are **mode-dependent**: they switch on/off depending on the contact mode $\sigma(t)$. For each foot $i \in \{1, 2, 3, 4\}$ (LF, RF, LH, RH), define:

$$c_i(t) = \begin{cases} 1 & \text{foot } i \text{ is in stance (contact)} \\ 0 & \text{foot } i \text{ is in swing} \end{cases} \quad (20)$$

5.1 Friction Cone Constraint

Active when: $c_i(t) = 1$ (stance).

For each stance foot i , the contact force $\mathbf{f}_i = [F_x, F_y, F_z]^\top$ must lie within the Coulomb friction cone:

$$\boxed{h_i^{\text{fric}}(\mathbf{u}) = \mu(F_z + F_{\text{grip}}) - \sqrt{F_x^2 + F_y^2} + \varepsilon_{\text{reg}} \geq 0} \quad (21)$$

where:

- $\mu = 0.5$: friction coefficient
- $F_{\text{grip}} = 0$: additional gripper force (zero for standard feet)
- $\varepsilon_{\text{reg}} = 25.0$: regularization to avoid gradient singularity at $\mathbf{f}_{\text{tan}} = \mathbf{0}$

Derivatives:

$$\frac{\partial h_i^{\text{fric}}}{\partial \mathbf{f}_i} = \begin{bmatrix} -F_x/\tilde{F}_t \\ -F_y/\tilde{F}_t \\ \mu \end{bmatrix}, \quad \tilde{F}_t = \sqrt{F_x^2 + F_y^2 + \varepsilon_{\text{reg}}} \quad (22)$$

$$\frac{\partial^2 h_i^{\text{fric}}}{\partial \mathbf{f}_i^2} = \begin{bmatrix} -(F_y^2 + \varepsilon_{\text{reg}})/\tilde{F}_t^3 & F_x F_y/\tilde{F}_t^3 & 0 \\ F_x F_y/\tilde{F}_t^3 & -(F_x^2 + \varepsilon_{\text{reg}})/\tilde{F}_t^3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (23)$$

5.1.1 Soft Constraint Formulation (Default)

When using the soft constraint formulation (default in the codebase), the friction cone inequality is converted to a penalty term added to the cost, using the **relaxed log-barrier** function:

$$p_{\text{friction},i}(\mathbf{u}) = \begin{cases} -\mu_b \ln(h_i^{\text{fric}}) & \text{if } h_i^{\text{fric}} > \delta_b \\ \mu_b \left[\frac{1}{2} \left(\frac{h_i^{\text{fric}} - 2\delta_b}{\delta_b} \right)^2 - \ln(\delta_b) \right] & \text{if } h_i^{\text{fric}} \leq \delta_b \end{cases} \quad (24)$$

with barrier parameters $\mu_b = 0.1$, $\delta_b = 5.0$. This provides a smooth C^1 penalty that becomes increasingly steep as $h \rightarrow 0^+$, while avoiding the singularity of the pure log barrier.

5.2 Zero Force Constraint (Swing Phase)

Active when: $c_i(t) = 0$ (swing).

Swing feet cannot exert contact forces:

$$\boxed{\mathbf{g}_i^{\text{zf}}(\mathbf{u}) = \mathbf{f}_i = \mathbf{0}_3} \quad (25)$$

This is enforced as an **equality constraint**.

5.3 Zero Velocity Constraint (Stance Phase)

Active when: $c_i(t) = 1$ (stance).

Stance feet must not slide on the ground:

$$\boxed{\mathbf{g}_i^{\text{zv}}(\mathbf{x}, \mathbf{u}) = \mathbf{A}_v \dot{\mathbf{p}}_{\text{foot},i}(\mathbf{x}, \mathbf{u}) + \mathbf{A}_x \mathbf{p}_{\text{foot},i}(\mathbf{x}) + \mathbf{b} = \mathbf{0}_3} \quad (26)$$

where:

$$\mathbf{A}_v = \mathbf{I}_3, \quad \mathbf{A}_x = \text{diag}(0, 0, \alpha_p), \quad \mathbf{b} = \mathbf{0}_3 \quad (27)$$

with α_p being the position error gain (set to 0 in the default configuration).

When $\alpha_p = 0$, this simplifies to:

$$\dot{\mathbf{p}}_{\text{foot},i} = \mathbf{0}_3 \quad (28)$$

which enforces that the foot velocity is zero in all three directions during stance — the **no-slip** condition.

The foot velocity is computed via the contact Jacobian:

$$\dot{\mathbf{p}}_{\text{foot},i} = \mathbf{J}_i(\mathbf{q}_{\text{gen}}) \dot{\mathbf{q}}_{\text{gen}} \quad (29)$$

5.4 Normal Velocity Constraint (Swing Phase)

Active when: $c_i(t) = 0$ (swing).

The vertical velocity of swing feet is constrained to follow the planned swing trajectory:

$$g_i^{\text{nv}}(\mathbf{x}, \mathbf{u}, t) = \dot{z}_{\text{foot},i} + \alpha_p(z_{\text{foot},i} - z_i^{\text{ref}}(t)) + \dot{z}_i^{\text{ref}}(t) = 0 \quad (30)$$

where $z_i^{\text{ref}}(t)$ and $\dot{z}_i^{\text{ref}}(t)$ come from the swing trajectory planner (Section 8).

This is a scalar equality constraint that guides the foot's vertical motion through the planned swing arc.

6 Constraint Activation Summary

Table 1 summarizes which constraints are active for each foot depending on its contact state.

Table 1: Constraint activation per foot.

Constraint	Stance ($c_i = 1$)	Swing ($c_i = 0$)
Friction cone (ineq/soft)	✓	—
Zero force (eq)	—	✓
Zero velocity (eq, 3D)	✓	—
Normal velocity (eq, 1D)	—	✓

7 Frontend: Gait Scheduling

The gait scheduler is the primary **frontend** component. It determines the contact mode sequence $\sigma(t)$ over the MPC horizon *before* the MPC optimization is solved.

7.1 Contact Mode Encoding

Each mode $\sigma \in \{0, 1, \dots, 15\}$ encodes the contact state of all 4 feet as a 4-bit number:

$$\sigma = 8 \cdot c_{\text{LF}} + 4 \cdot c_{\text{RF}} + 2 \cdot c_{\text{LH}} + 1 \cdot c_{\text{RH}} \quad (31)$$

Key modes:

σ	Description
15	STANCE: all 4 feet in contact
9	LF + RH: trot diagonal pair
6	RF + LH: trot diagonal pair
0	FLY: no feet in contact

7.2 Gait Definition

A gait is defined as a periodic template:

$$\mathcal{G} = (T_{\text{gait}}, \{\phi_1, \dots, \phi_{K-1}\}, \{\sigma_0, \sigma_1, \dots, \sigma_{K-1}\}) \quad (32)$$

where:

- T_{gait} : gait cycle period [s]

- $\phi_k \in (0, 1)$: phase at which the k -th mode transition occurs
- σ_k : contact mode during the k -th phase interval

For example, a trotting gait might be:

$$\mathcal{G}_{\text{trot}} = (0.6, \{0.5\}, \{9, 6\})$$

meaning: period 0.6 s, switching at phase 0.5, alternating between mode 9 (LF+RH stance) and mode 6 (RF+LH stance).

7.3 Mode Schedule Generation

Given the current time t_0 and horizon T , the gait scheduler:

1. Takes the current gait template $\mathcal{G}$
2. **Tiles** the template repeatedly to cover $[t_0, t_0 + T]$
3. Inserts a brief **stance transition phase** (duration $t_{\text{trans}} = 0.4$ s) at every gait switch point to ensure smooth transitions
4. Outputs the **mode schedule**: a piecewise-constant function $\sigma : [t_0, t_0 + T] \rightarrow \{0, \dots, 15\}$ with switching times $\{t_1^s, t_2^s, \dots\}$

Algorithm 1 Mode Schedule Generation (`GaitSchedule::getModeSchedule`)

- 1: **Input:** current time t , horizon end $t + T$, gait template $\mathcal{G}$
 - 2: Trim past events from schedule
 - 3: If current end mode $\neq$ STANCE: insert stance phase of duration t_{trans}
 - 4: **while** schedule does not cover $t + T$ **do**
 - 5: **for** each phase in gait template **do**
 - 6: Append mode and compute absolute event time from phase
 - 7: **end for**
 - 8: **end while**
 - 9: **Output:** event times $\{t_k^s\}$ and modes $\{\sigma_k\}$
-

7.4 Reference Manager

The `SwitchedModelReferenceManager` coordinates:

1. Updating the mode schedule over the MPC horizon (from gait scheduler)
2. Updating swing foot trajectories (from swing planner, Section 8)
3. Providing contact flags $c_i(t)$ to all mode-dependent constraints

At each MPC iteration, it calls:

$$\text{modifyReferences}(t_0, t_0 + T, \mathbf{x}_0, \text{targetTrajectories}, \text{modeSchedule})$$

8 Frontend: Swing Trajectory Planning

For each swing phase, a smooth foot trajectory in the vertical (z) direction is planned using cubic splines. This trajectory serves as a reference for the normal velocity constraint (Section 5.4).

8.1 Swing Phase Identification

Given the mode schedule, for each foot i , the planner identifies swing intervals $[t_{lo}, t_{td}]$ where:

- t_{lo} : lift-off time (transition from $c_i = 1$ to $c_i = 0$)
- t_{td} : touch-down time (transition from $c_i = 0$ to $c_i = 1$)

8.2 Vertical Trajectory Construction

A cubic spline $z_i^{\text{ref}}(t)$ is constructed with three nodes:

- **Lift-off node:** $(t_{lo}, z_{lo}, s \cdot v_{lo})$
- **Mid-swing peak:** $z_{\text{mid}} = \min(z_{lo}, z_{td}) + s \cdot h_{\text{swing}}$
- **Touch-down node:** $(t_{td}, z_{td}, s \cdot v_{td})$

where the scaling factor adapts to short swing phases:

$$s = \min\left(1, \frac{t_{td} - t_{lo}}{t_{\text{scale}}}\right) \quad (33)$$

Configuration parameters:

Parameter	Value	Description
v_{lo}	0.2 m/s	Initial vertical lift-off velocity
v_{td}	-0.4 m/s	Final vertical touch-down velocity
h_{swing}	0.1 m	Maximum swing height above terrain
t_{scale}	0.15 s	Minimum reference swing duration

The resulting trajectory provides $z_i^{\text{ref}}(t)$ and $\dot{z}_i^{\text{ref}}(t)$ used in the normal velocity constraint (Section 5.4).

9 Frontend: Pre-Computation and Dynamic Constraint Updates

The `LeggedRobotPreComputation` module runs before each constraint/cost evaluation and dynamically configures time-varying constraint parameters:

1. Reads current contact flags $c_i(t)$ from the reference manager
2. For each swing foot, queries the swing trajectory planner:

$$\dot{z}_i^{\text{ref}}(t) \leftarrow \text{getZvelocityConstraint}(i, t) \quad (34)$$

$$z_i^{\text{ref}}(t) \leftarrow \text{getZpositionConstraint}(i, t) \quad (35)$$

3. Updates the normal velocity constraint configuration:

$$\mathbf{b} = -\dot{z}_i^{\text{ref}}(t) \quad (36)$$

$$\mathbf{A}_x = \alpha_p \cdot [0, 0, 1]^\top, \quad \text{adjusted by } z_i^{\text{ref}}(t) \quad (37)$$

This mechanism allows constraints to be time-varying (tracking swing trajectories) while using a static constraint interface.

10 Solver Configuration

10.1 Multiple Shooting Transcription

The continuous-time OCP is discretized using multiple shooting with $N \approx 67$ nodes:

$$t_k = t_0 + k \cdot \Delta t, \quad \Delta t = 0.015 \text{ s}, \quad k = 0, \dots, N \quad (38)$$

At each node, the dynamics are integrated using a 2nd-order Runge-Kutta (RK2) scheme:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \cdot f\left(\mathbf{x}_k + \frac{\Delta t}{2} f(\mathbf{x}_k, \mathbf{u}_k), \mathbf{u}_k\right) \quad (39)$$

10.2 SQP Solver (Default)

The default solver is Sequential Quadratic Programming with:

Parameter	Value	Description
SQP iterations	1	Single QP solve per MPC cycle
Threads	3	Parallel shooting
Δt	0.015 s	Discretization step
Integrator	RK2	2nd-order Runge-Kutta
Feedback policy	true	Returns linear feedback gains
Constraint tolerance	10^{-4}	Convergence criterion

Since only 1 SQP iteration is performed per MPC cycle, the solver relies on warm-starting from the previous solution. The MPC runs at 50 Hz with a multi-rate thread at 400 Hz for interpolating the policy.

10.3 Alternative Solvers

- **DDP (SLQ variant):** Uses iterative LQ approximation with line search. Soft constraints via augmented Lagrangian penalty ($\lambda_0 = 20.0$, growth rate 2.0).
- **IPM:** Interior point method with barrier parameter schedule (initial 10^{-4} , target 10^{-4}).

11 Complete Discretized OCP

Collecting all components, the discretized problem solved at each MPC cycle is:

$$\begin{aligned}
 & \min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \left[\|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_Q^2 + \|\mathbf{u}_k - \mathbf{u}_k^{\text{ref}}\|_R^2 + \sum_{i: c_i(t_k)=1} p_{\text{fric},i}(\mathbf{u}_k) \right] + \|\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\|_Q^2 \\
 & \text{s.t.} \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \cdot f_{\text{RK2}}(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1 \\
 & \quad \dot{\mathbf{p}}_{\text{foot},i}(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{0}, \quad \forall i : c_i(t_k) = 1 \quad (\text{no-slip}) \\
 & \quad \mathbf{f}_i = \mathbf{0}, \quad \forall i : c_i(t_k) = 0 \quad (\text{zero force}) \\
 & \quad \dot{z}_{\text{foot},i} = -\dot{z}_i^{\text{ref}}(t_k) - \alpha_p(z_{\text{foot},i} - z_i^{\text{ref}}(t_k)), \quad \forall i : c_i(t_k) = 0 \quad (\text{swing traj.}) \\
 & \quad \mu F_{z,i} \geq \sqrt{F_{x,i}^2 + F_{y,i}^2} + \varepsilon, \quad \forall i : c_i(t_k) = 1 \quad (\text{friction, if hard}) \\
 & \quad \mathbf{x}_0 = \hat{\mathbf{x}}(t_0)
 \end{aligned} \quad (40)$$

12 Summary of the Full Pipeline

The complete legged robot MPC pipeline consists of the following components running in a loop:

1. **State Estimation** (external): Provides current state $\hat{\mathbf{x}}(t_0)$
2. **Gait Scheduler** (frontend):
 - Selects gait template (trot, walk, pace, etc.) based on user command
 - Tiles the template over $[t_0, t_0 + T]$
 - Outputs mode schedule $\sigma(t)$ and event times
3. **Reference Manager** (frontend):
 - Generates target state trajectory $\mathbf{x}^{\text{ref}}(t)$ from velocity commands
 - Computes weight-compensating input reference $\mathbf{u}^{\text{ref}}(t)$
 - Extracts per-foot contact flags $c_i(t)$ from mode schedule
4. **Swing Trajectory Planner** (frontend):
 - For each swing phase, plans vertical foot trajectory via cubic spline
 - Provides $z_i^{\text{ref}}(t)$, $\dot{z}_i^{\text{ref}}(t)$ to the normal velocity constraint
5. **MPC Solver** (core):
 - Solves the OCP (Section 11) via SQP/DDP/IPM
 - Outputs optimal trajectory $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$ and (optionally) feedback gains
6. **Whole-Body Controller** (downstream, not part of OCS2):
 - Maps centroidal-level commands $(\mathbf{f}_i, \dot{\mathbf{q}}_j)$ to actual joint torques
 - Typically a QP-based inverse dynamics or task-space controller

13 Key Design Decisions and Notes

- **Contact is NOT implicit.** Unlike contact-implicit trajectory optimization (e.g., complementarity-based methods), OCS2 relies on a predetermined mode schedule. The gait scheduler decides *when* each foot is in stance/swing. The MPC does not optimize over contact timing.
- **Foothold placement is implicit.** While the contact *timing* is fixed, the contact *location* (where the foot lands) is not explicitly planned. Instead, it emerges from the optimization: the zero-velocity constraint pins the foot during stance, and the swing trajectory guides the foot in z , but the x, y landing position is determined by the optimizer minimizing the cost.
- **SRBD vs. Full Centroidal.** The default SRBD model keeps the centroidal inertia constant, which is faster but less accurate for robots with heavy limbs. Setting `centroidalModelType = 0` enables full centroidal dynamics where $\mathbf{A}_g$ is recomputed at every configuration.
- **CppAD automatic differentiation.** Constraints and dynamics are differentiated automatically using CppAD. The generated libraries are cached in `/tmp/ocs2` and recompiled when model parameters change.

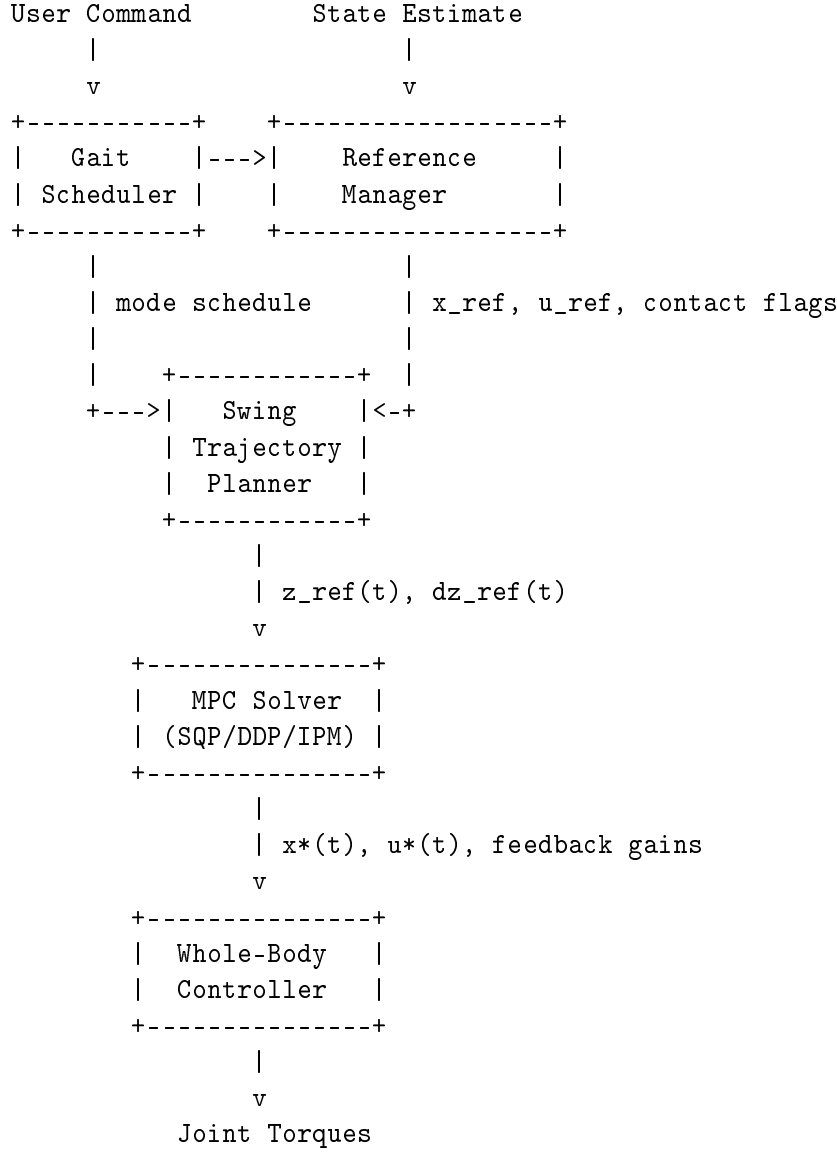


Figure 1: OCS2 Legged Robot MPC Pipeline

- **No joint torque limits in MPC.** The MPC operates in the force/velocity space and does not enforce actuator torque limits. These must be handled by the downstream whole-body controller.
- **Flat terrain assumption.** The basic legged robot example assumes flat ground at $z = 0$. The advanced perceptive locomotion example (`ocs2_perceptive_anymal`) extends this to uneven terrain.